Correction de TD2

Exercice1 :

```
#include <stdio.h>
int main()
{
 /* Déclarations */
 int A[50]; /* tableau donné    */
 int VAL;   /* valeur à insérer */
 int N;     /* dimension        */
 int I;     /* indice courant   */

 /* Saisie des données */
 printf("Dimension N du tableau initial (max.50) : ");
 scanf("%d", &N );
 for (I=0; I<N; I++)
    {
     printf("Elément %d : ", I);
     scanf("%d", &A[I]);
    }
 printf("Elément à insérer : ");
 scanf("%d", &VAL );
 /* Affichage du tableau */
 printf("Tableau donné : \n");
 for (I=0; I<N; I++)
    printf("%d ", A[I]);
 printf("\n");
 /* Déplacer les éléments plus grands que */
```

```
 /* VAL d'une position vers l'arrière.    */
 for (I=N ; (I>0)&&(A[I-1]>VAL) ; I--)
       A[I]=A[I-1];
 /* VAL est copié à la position du dernier */
 /* élément déplacé. */
 A[I]=VAL;
 /* Nouvelle dimension du tableau ! */
 N++;
  /* Edition des résultats */
 printf("Tableau résultat :\n");
 for (I=0; I<N; I++)
    printf("%d ", A[I]);
 printf("\n");
 return 0;
}
```

Exercice 2 :

```
#include <stdio.h>
int main()
{
 /* Déclarations */
 int A[50][50];  /* matrice initiale */
 int B[50][50];  /* matrice résultat */
 int N, M;       /* dimensions des matrices */
 int I, J;       /* indices courants */

 /* Saisie des données */
 printf("Nombre de lignes   (max.50) : ");
 scanf("%d", &N );
```

```
printf("Nombre de colonnes (max.50) : ");
scanf("%d", &M );
for (I=0; I<N; I++)
  for (J=0; J<M; J++)
      {
       printf("Elément[%d][%d] : ",I,J);
       scanf("%d", &A[I][J]);
      }
/* Affichage de la matrice */
printf("Matrice donnée :\n");
for (I=0; I<N; I++)
  {
   for (J=0; J<M; J++)
       printf("%7d", A[I][J]);
   printf("\n");
  }
/* Affectation de la matrice transposée à B */
for (I=0; I<N; I++)
   for (J=0; J<M; J++)
       B[J][I]=A[I][J];
 /* Edition du résultat */
/* Attention: maintenant le rôle de N et M est inversé. */
 printf("Matrice résultat :\n");
for (I=0; I<M; I++)
  {
   for (J=0; J<N; J++)
       printf("%7d", B[I][J]);
   printf("\n");
```

```
   }
 return 0;
}
```

Exercice 3 :

```
#include <stdio.h>
main()
{
 /* Déclarations */
 int A[50][50]; /* matrice donnée   */
 int B[50][50]; /* matrice donnée   */
 int C[50][50]; /* matrice résultat */
 int N, M;      /* dimensions des matrices */
 int I, J;      /* indices courants */

 /* Saisie des données */
 printf("Nombre de lignes   (max.50) : ");
 scanf("%d", &N );
 printf("Nombre de colonnes (max.50) : ");
 scanf("%d", &M );
 printf("*** Matrice A ***\n");
 for (I=0; I<N; I++)
   for (J=0; J<M; J++)
       {
        printf("Elément[%d][%d] : ",I,J);
        scanf("%d", &A[I][J]);
       }
 printf("*** Matrice B ***\n");
 for (I=0; I<N; I++)
```

```
   for (J=0; J<M; J++)
       {
        printf("Elément[%d][%d] : ",I,J);
        scanf("%d", &B[I][J]);
       }
/* Affichage des matrices */
printf("Matrice donnée A :\n");
for (I=0; I<N; I++)
   {
    for (J=0; J<M; J++)
         printf("%7d", A[I][J]);
    printf("\n");
   }
printf("Matrice donnée B :\n");
for (I=0; I<N; I++)
   {
    for (J=0; J<M; J++)
         printf("%7d", B[I][J]);
    printf("\n");
   }

/* Affectation du résultat de l'addition à C */
for (I=0; I<N; I++)
    for (J=0; J<M; J++)
          C[I][J] = A[I][J]+B[I][J];
 /* Edition du résultat */
printf("Matrice résultat C :\n");
for (I=0; I<N; I++)
```

```
    {
     for (J=0; J<M; J++)
          printf("%7d", C[I][J]);
     printf("\n");
    }
 return 0;
}
```

Exercice 4 :

```
#include <stdio.h>
main()
{
 /* Déclarations */
 int A[50][50]; /* matrice donnée   */
 int B[50][50]; /* matrice donnée   */
 int C[50][50]; /* matrice résultat */
 int N, M, P;   /* dimensions des matrices */
 int I, J, K;   /* indices courants */

 /* Saisie des données */
 printf("*** Matrice A ***\n");
 printf("Nombre de lignes de   A (max.50) : ");
 scanf("%d", &N );
 printf("Nombre de colonnes de A (max.50) : ");
 scanf("%d", &M );
 for (I=0; I<N; I++)
    for (J=0; J<M; J++)
        {
         printf("Elément[%d][%d] : ",I,J);
```

```
        scanf("%d", &A[I][J]);
       }
printf("*** Matrice B ***\n");
printf("Nombre de lignes de   B : %d\n", M);
printf("Nombre de colonnes de B (max.50) : ");
scanf("%d", &P );
for (I=0; I<M; I++)
   for (J=0; J<P; J++)
       {
        printf("Elément[%d][%d] : ",I,J);
        scanf("%d", &B[I][J]);
       }
/* Affichage des matrices */
printf("Matrice donnée A :\n");
for (I=0; I<N; I++)
   {
    for (J=0; J<M; J++)
         printf("%7d", A[I][J]);
    printf("\n");
   }
printf("Matrice donnée B :\n");
for (I=0; I<M; I++)
   {
    for (J=0; J<P; J++)
         printf("%7d", B[I][J]);
    printf("\n");
   }
/* Affectation du résultat de la multiplication à C */
```

```
for (I=0; I<N; I++)
    for (J=0; J<P; J++)
        {
         C[I][J]=0;
         for (K=0; K<M; K++)
             C[I][J] += A[I][K]*B[K][J];
        }
 /* Edition du résultat */
printf("Matrice résultat C :\n");
for (I=0; I<N; I++)
   {
    for (J=0; J<P; J++)
        printf("%7d", C[I][J]);
    printf("\n");
   }
return 0;
}
```

Exercice 5 :

```
#include <stdio.h>
#include <string.h>
int main()
{
 char T[100];
 int freq[100];
 int i,j,compteur;
 int max,indice_max;
 puts("veuillez enter une chaine: ");
 gets(T);
```

```
for(i=0; i<strlen(T); i++){
      freq[i]=-1;
}
for(i=0; i<strlen(T); i++){
    compteur=1;
    for(j=i+1; j<strlen(T); j++){
        if(T[i]==T[j]){
            compteur ++;
            freq[j]=0;
        }
    }
    if(freq[i]!=0){
        freq[i]=compteur;
    }
}
max=freq[0];
indice_max=0;
for(i=1; i<strlen(T); i++){
    if(freq[i]>max){
        max=freq[i];
        indice_max=i;
    }
}
printf("le caractere qui se repete le plus dans la chaine est: %c",T[indice_max]);
 return 0;
}
```

Exercice 6 :

```
#include <stdio.h>
void main()
{
 /* Déclarations */
 int T[50]; /* tableau donné */
 int N; /* dimension */
 int I,J; /* indices courants */
 int AIDE; /* pour l'échange */
 /* Saisie des données */
 printf("Dimension du tableau (max.50) : ");
 scanf("%d", &N );
 for (I=0; I<N; I++)
 {
 printf("Elément %d : ", I);
 scanf("%d", &T[I]);
 }
 /* Affichage du tableau */
 printf("Tableau donné : \n");
 for (I=0; I<N; I++)
 printf("%d ", T[I]);
 printf("\n");
 /* Inverser le tableau */
 for (I=0, J=N-1 ; I<J ; I++,J--)
 /* Echange de T[I] et T[J] */
 {
 AIDE = T[I];
 T[I] = T[J];
 T[J] = AIDE;
```

```
}
/* Edition des résultats */
printf("Tableau résultat :\n");
for (I=0; I<N; I++)
printf("%d ", T[I]);
printf("\n");
}
```

Exercice 7 :

```
#include <stdio.h>
void main()
{
 /* Déclarations */
 int T[50]; /* tableau donné */
 int N; /* dimension */
 int I,J; /* indices courants */
 /* Saisie des données */
 printf("Dimension du tableau (max.50) : ");
 scanf("%d", &N );
 for (I=0; I<N; I++)
 {
 printf("Elément %d : ", I);
 scanf("%d", &T[I]);
 }
 /* Affichage du tableau */
 printf("Tableau donné : \n");
 for (I=0; I<N; I++)
 printf("%d ", T[I]);
 printf("\n");
```

```
 /* Effacer les zéros et comprimer : */
 /* Copier tous les éléments de I vers J et */
 /* augmenter J pour les éléments non nuls. */
 for (I=0, J=0 ; I<N ; I++)
 {
T[J] = T[I];
if (T[I]) J++;
 }
 /* Nouvelle dimension du tableau ! */
 N = J;
 /* Edition des résultats */
 printf("Tableau résultat :\n");
 for (I=0; I<N; I++)
 printf("%d ", T[I]);
 printf("\n");
}
```

Exercice 8 :

```
#include <stdio.h>
int main()
{
 /* Déclarations */
 char SUJ[100]; /* chaîne à transformer        */
 char OBJ[100]; /* chaîne à supprimer dans SUJ */
 int I;         /* indice courant dans SUJ     */
 int J;         /* indice courant dans OBJ     */
 int TROUVE;    /* indicateur logique qui précise */
                /* si la chaîne OBJ a été trouvée */
```

```
/* Saisie des données */
printf("Introduisez la chaine à supprimer   : ");
gets(OBJ);
printf("Introduisez la chaine à transformer : ");
gets(SUJ);
/* Recherche de OBJ dans SUJ */
TROUVE=0;
for (I=0; SUJ[I] && !TROUVE; I++)
   /* Si la première lettre est identique, */
   if (SUJ[I]==OBJ[0])
       {
        /* alors comparer le reste de la chaîne */
        for (J=1; OBJ[J] && (OBJ[J]==SUJ[I+J]); J++)
          ;
        if (OBJ[J]=='\0') TROUVE=1;
       }
/* Si la position de départ de OBJ dans SUJ a été trouvée */
/* alors déplacer le reste de SUJ à cette position. */
if (TROUVE)
   {
    I--;
    /* Maintenant  I indique la position de OBJ */
    /* dans SUJ et J indique la longueur de OBJ */
    for (; SUJ[I+J]; I++)
        SUJ[I]=SUJ[I+J];
    SUJ[I]='\0';
   }
 /* Affichage du résultat */
```

```
 printf("Chaine resultat : \"%s\"\n", SUJ);
 return 0;
}
```